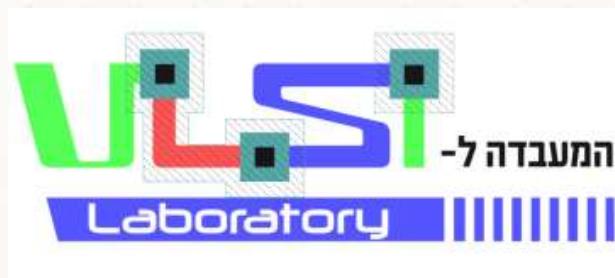




Technion
Israel Institute of Technology



The Andrew and Erna Viterbi Faculty of
**ELECTRICAL & COMPUTER
ENGINEERING**

Full Chip Verification Environment

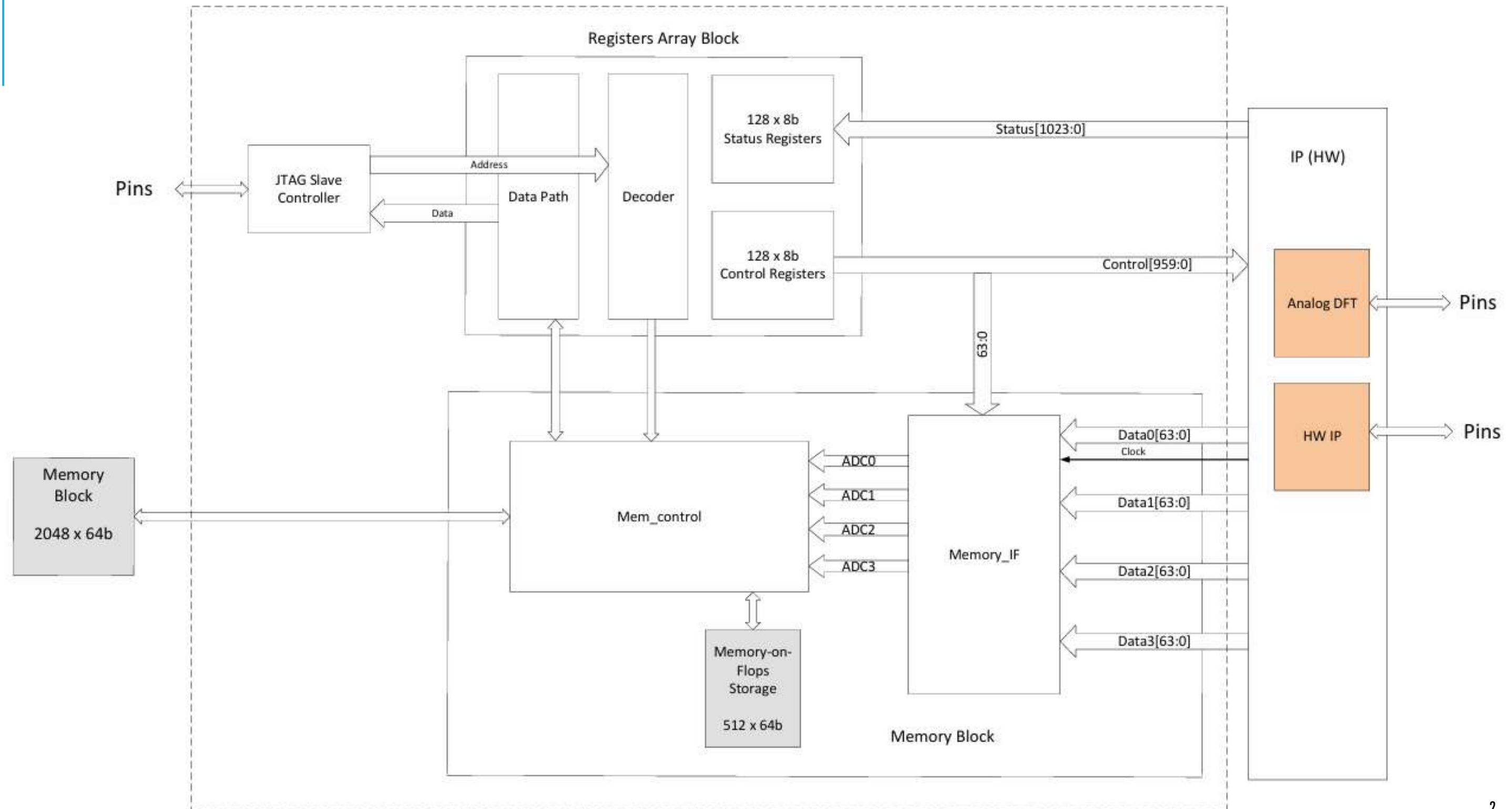
PROJECT B

Students: Samer Tabri, Mohammed Hosin

Supervisor: Yehoshua Raanan

Winter Semester 2025/2026

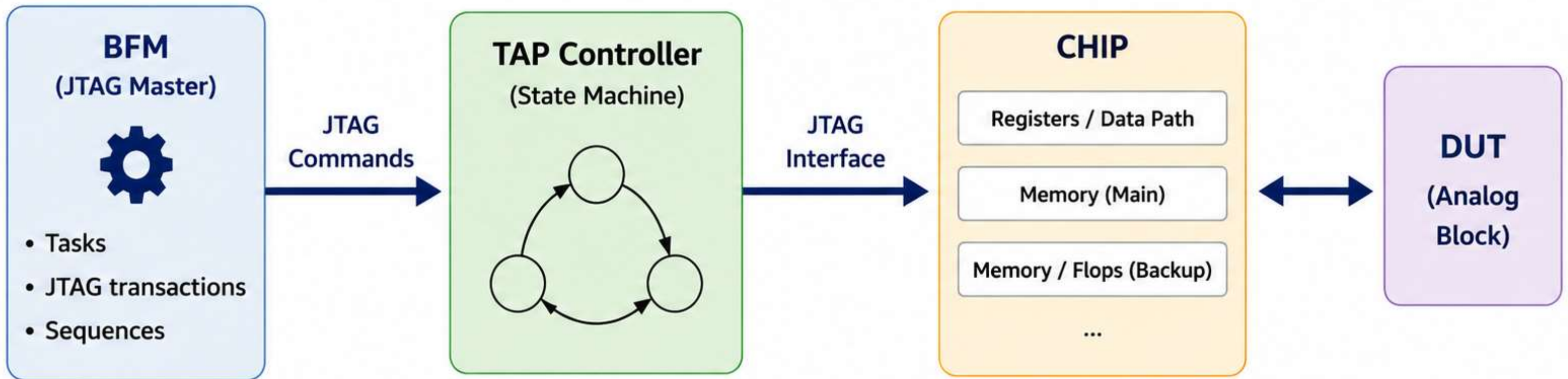
FULL CHIP – HIGH LEVEL OVERVIEW



3. Verification Flow – BFM, TAP Controller and Tests

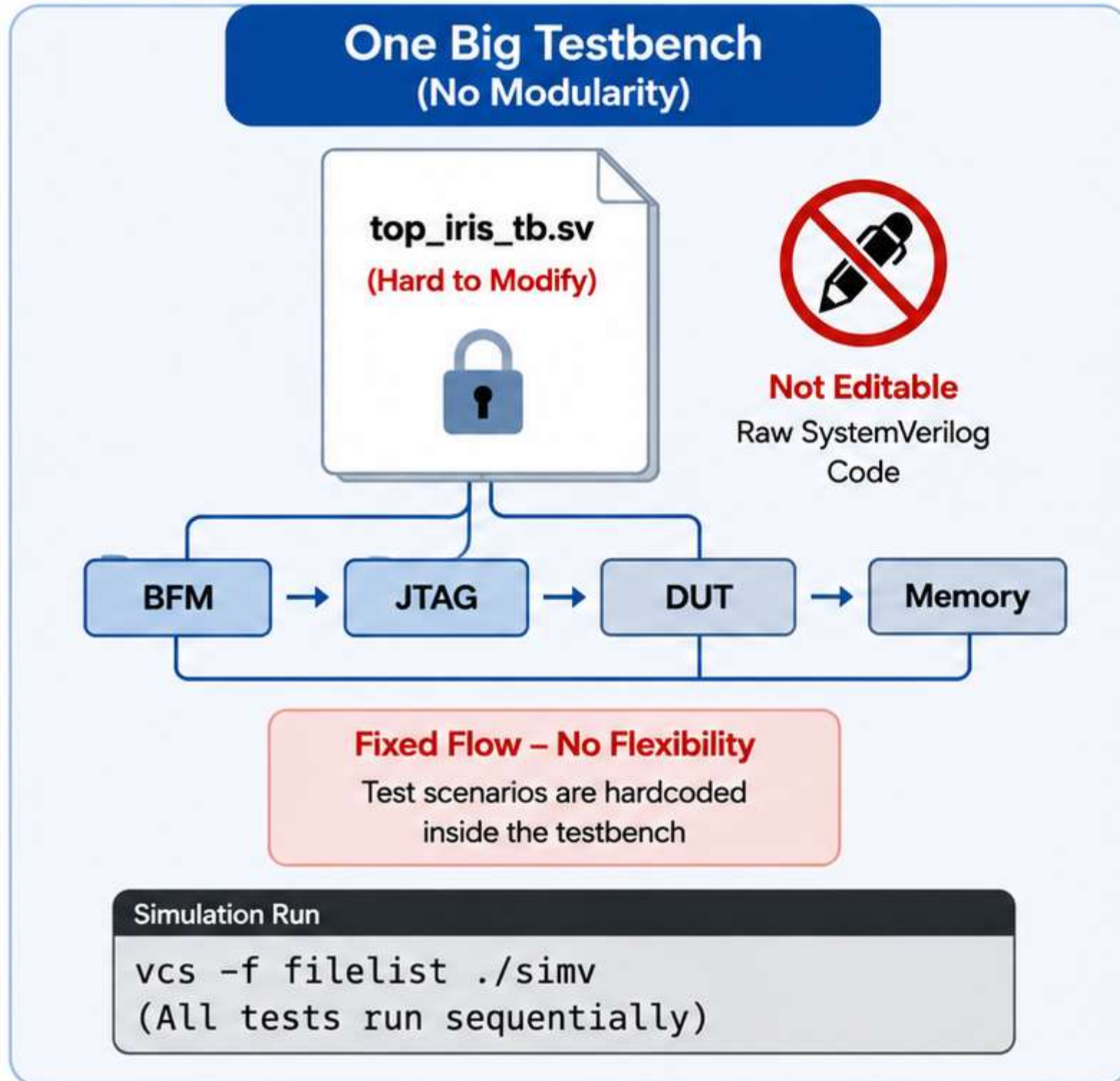
We verify the CHIP by sending JTAG commands through the TAP Controller.

The **BFM** (JTAG Master) uses **Tasks** to create JTAG transactions and check the CHIP behavior.



The Problem with the Old Verification Approach

Why We Needed a New Framework



Single Big Testbench

All tests are embedded inside a single testbench environment; no modular test isolation.



Zero User Control

Impossible to select or isolate specific test scenarios without manually editing raw SystemVerilog code.



Simulation Drag

Execution is forced into one long, inefficient simulation run, making debug tracing slow and difficult.

Simple vs. Advanced: Manual Scripting vs. Automated Orchestration

	Simple (Manual) Approach		Advanced (Automated) Approach
Test Selection	Tests are written by hand using SystemVerilog tasks.	➡	Dynamic and selective via Python CLI or Web GUI .
Execution Engine	A fixed, unchangeable simulator sequence.	➡	Python backend orchestrating VCS compilation and execution.
Testbench Structure	A single large file. top_iris_tb.sv	➡	Modular , separate test blocks injected dynamically into a temporary setup.
User Knowledge	Requires in-depth SystemVerilog knowledge.	➡	Zero SystemVerilog knowledge required to run, configure, or monitor regressions.



Normal (Manual) Approach – Example: READ / WRITE Test

Limitations of the Normal Approach

- **Tight coupling:** Test logic is directly embedded in the testbench.
- **Hard to maintain:** Each new test requires adding many lines of code and tasks.
- **Limited reuse:** Common sequences are rewritten across multiple tests.
- **Low scalability:** As the number of tests grows, the testbench becomes complex and hard to manage.
- **No flexibility:** Cannot easily change parameters or run selectively.
- **Debugging difficulty:** Failures are harder to isolate due to long and mixed flows.

```
//Test No. 13/19 - Only 1 buffer fill; Partial buffer.
`ifdef MEMORY_ON_FLOPS
setup_to_memory_on_flops_function();
flops_tests_index = 6;
partial_address = 11'd22;
`else
flops_tests_index = 0;
partial_address = 11'd200;
`endif
setup_for_general_startup (13+flops_tests_index, TRUE);
tested_buffer_size = partial_address;
setup_to_fill_memory_partial_buffer_till_address(partial_address);
setup_to_fill_memory_1_full_buffer_only();
start_gathering_data ();
fork
    ip_generate_random_data_endlessy_or_x_times (partial_address+20, FALSE, FALSE, 3'b000, 12'h000);
    stop_gathering_data (3000);
join
setup_to_host_access_memory();
host_read_full_memory_buffer(read_buffer, data_out);
compare_buffers(tested_buffer_size, 13+flops_tests_index, FALSE);
```




Separate Approach – Solution: Each Test as a Task

Advantages of the Separate Approach

- ✓ **Modular and organized:**
Each test is implemented as a separate task with a clear purpose.
- ✓ **Easy to maintain:**
Adding, updating or debugging a test does not affect others.
- ✓ **Reusability:** Common tasks and BFM functions are reused across tests.
- ✓ **Scalability:**
New tests can be added easily without increasing complexity.

Project_B > rev12 > functions > tests_separate >

test_01_check_serial_output_po sv

test_02_read_all_control_registers_po_values sv

test_03_write_read_all_control_registers sv

test_04_read_all_control_registers sv

test_05_masked_write_read_all_control_registers sv

test_06_write_read_all_status_registers sv

test_07_read_all_status_registers sv

test_08_hw_write_read_all_status_registers sv

test_10_16_write_read_dead_beef_from_host sv

test_11_17_write_read_full_buffer_from_host sv

test_12_18_hw_write_one_full_buffer sv

test_13_19_hw_write_one_partial_buffer sv

test_14_20_hw_write_endless_full_buffer sv

test_15_21_hw_write_endless_partial_buffer sv

test_22_26_hw_write_ds_one_full_buffer sv

test_23_27_hw_write_ds_one_partial_buffer sv

test_24_28_hw_write_ds_endless_full_buffer sv

test_25_29_hw_write_ds_endless_partial_buffer sv

test_30_34_hw_write_4adcs_one_full_buffer sv

test_31_35_hw_write_4adcs_one_partial_buffer sv

test_32_36_hw_write_4adcs_endless_full_buffer sv

test_33_37_hw_write_4adcs_endless_partial_buffer sv

test_38_42_hw_write_4adcs_ds_one_full_buffer sv

test_39_43_hw_write_4adcs_ds_one_partial_buffer sv

test_40_44_hw_write_4adcs_ds_endless_full_buffer sv

test_41_45_hw_write_4adcs_ds_endless_partial_buffer sv

tests_include sv

```

// Test No. 2 - Read all Control registers and compare to the P0 default values.
// task read_all_control_registers_po_values(output [7:0] returned_data, input [0:127][7:0] expected_data, input boolean verbose);
boolean pass;
pass = TRUE;
for (int a = 0 ; a < 128; a = a + 1) begin
    logic [7:0] add;
    logic [7:0] returned_data;
    add = a [7:0];
    read_control_register(add, returned_data); // Read Control register

    // Now checking
    if (returned_data != expected_data[add]) begin
        $error("Unexpected returned data at address %02d: 0x%02h; expected: 0x%02h", add, returned_data, expected_data[add]);
        pass = FALSE;
    end else begin
        if (verbose) $display("Test No. 2 - succeeded at address: %02d", add);
    end
end
if (pass) $display("Test No. 2 - pasceed!"); else $display("Test No. 2 - failed!!");
endtask
    
```




Separate Approach – Implementation: Clean and Organized Testbench

How We Implemented the Solution

- 1 Created separate tasks for each test**
Each test was implemented as a dedicated task in its own file.
- 2 Created an include file for all tasks**
A central file (tests_include.sv) includes all the individual test task files.
- 3 Included the tasks in the testbench**
The testbench now includes only the central include file and the BFM.
- 4 Clean and organized testbench**
The testbench contains only a simple, clear sequence of test calls – easy to read, maintain and extend.

1 All tests implemented as separate tasks (one file per test)

```
tests_include.sv
~/Project_B/rev12/functions/tests_separate

include "/users/epstmh/Project_B/rev12/functions/tests_separate/test_01_check_serial_output_po.sv"
include "/users/epstmh/Project_B/rev12/functions/tests_separate/test_02_read_all_control_registers_po_values.sv"
include "/users/epstmh/Project_B/rev12/functions/tests_separate/test_03_write_read_all_control_registers.sv"
include "/users/epstmh/Project_B/rev12/functions/tests_separate/test_04_read_all_control_registers.sv"
include "/users/epstmh/Project_B/rev12/functions/tests_separate/test_05_masked_write_read_all_control_registers.sv"
include "/users/epstmh/Project_B/rev12/functions/tests_separate/test_06_write_read_all_status_registers.sv"
include "/users/epstmh/Project_B/rev12/functions/tests_separate/test_07_read_all_status_registers.sv"
include "/users/epstmh/Project_B/rev12/functions/tests_separate/test_08_hw_write_read_all_status_registers.sv"
include "/users/epstmh/Project_B/rev12/functions/tests_separate/test_10_l6_write_read_dead_beef_from_host.sv"
include "/users/epstmh/Project_B/rev12/functions/tests_separate/test_10_l16_write_read_dead_beef_from_host.sv"
include "/users/epstmh/Project_B/rev12/functions/tests_separate/test_11_17_write_read_full_buffer_from_host.sv"
...
```

2 Central include file that includes all task files

```
include "/users/epstmh/Project_B/rev12/functions/fc_function12.sv"
include "/users/epstmh/Project_B/rev12/functions/tests_separate/tests_include.sv"
```

3 The testbench includes only the central include and the BFM

4 Clean and organized testbench – simple test flow

```

//***** Serial / P0 Tests *****//
//***** Registers Tests *****//

// Test No. 1 - collect serial data automatically right after power-on.
check_serial_output_po(FALSE);

// Test No. 2 - Read all Control registers and compare to the P0 default values.
read_all_control_registers_po_values(returned_data, expected_data, FALSE);
// Test No. 3 - Write data (data = address) to all Control registers, then read back.
write_read_all_control_registers(returned_data, FALSE);
// Test No. 4 - Read all Control registers again, verify no address aliasing.
read_all_control_registers(returned_data, FALSE);

// Test No. 5 - Write with mask=0 (no write), read back, verify data unchanged.
masked_write_read_all_control_registers(returned_data, FALSE);

// Test No. 6 - Write all Status registers from host (in_sel=1), then read back.
write_read_all_status_registers(returned_data, FALSE);

// Test No. 7 - Read all Status registers again, verify no address aliasing.
read_all_status_registers(returned_data, FALSE);

// Test No. 8 - Write all Status registers from HW (in_sel=0), then read back.
hw_write_read_all_status_registers(returned_data, FALSE);
```


Python Framework – Phase 1: Basic Wrapper

Started with a Single Python Script

What We Implemented in Phase 1

- ✓ **Started with a single Python script**
A standalone Python file acts as the initial wrapper (no GUI or advanced features yet).
- ✓ **Responsible for running the testbench**
The script compiles and runs the SystemVerilog testbench.
- ✓ **Provides configuration and control**
Includes basic settings such as file paths, simulator options, and run parameters.
- ✓ **Displays simulation output**
Prints the results and logs from the simulation to the console.

Initial Python Script (Single File)

```
#!/usr/bin/env python3
import subprocess
import os

PROJECT_DIR = "/users/epstmh/Project_B/rev12"
FILELIST = os.path.join(PROJECT_DIR, "filelist")

vcs_cmd = [
    "vcs", "-kdb", "-sverilog", "-v2k_generate",
    "-debug_access+all", "-full64",
    "-ignore", "initializer_driver_checks",
    "+define+ARM_DISABLE_EMA_CHECK",
    "+define+VALIDATION",
    "+incdir+/users/shukir/logic_design/Project/design/work/include",
    "-f", FILELIST,
]

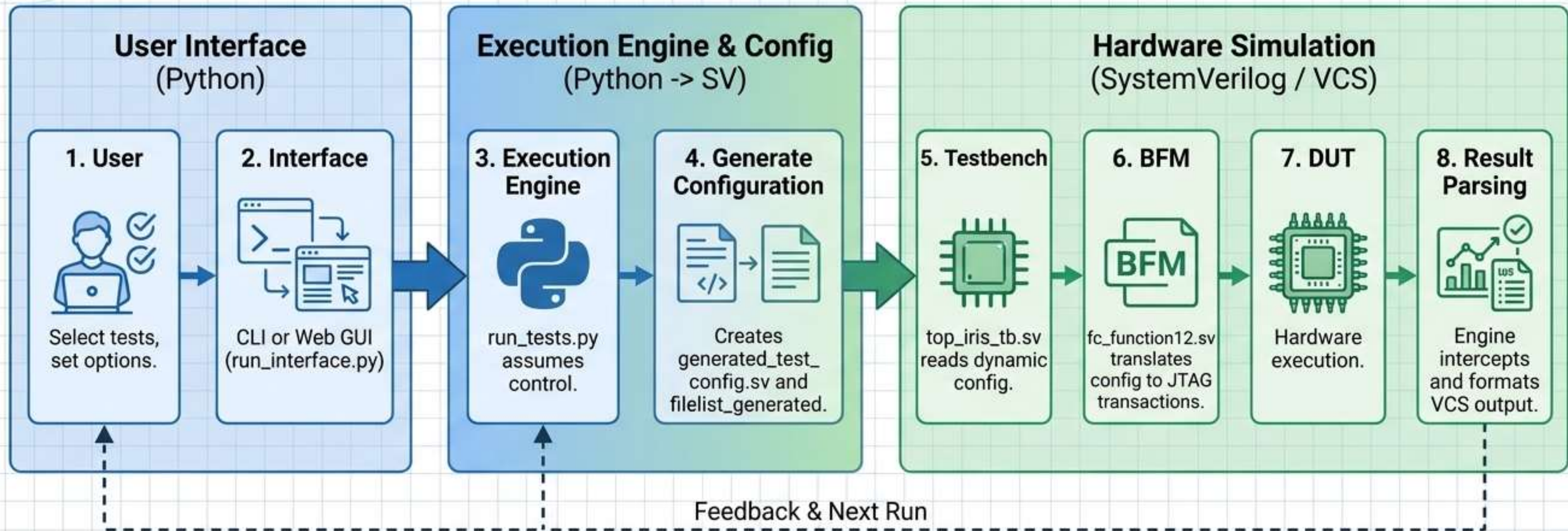
r = subprocess.run(vcs_cmd, cwd=PROJECT_DIR, stdout=subprocess.PIPE, stderr=subprocess.PIPE,
                    universal_newlines=True, timeout=300)
if r.returncode != 0:
    print(r.stdout)
    print(r.stderr)
    exit(1)

r = subprocess.run([os.path.join(PROJECT_DIR, "simv")], cwd=PROJECT_DIR,
                    stdout=subprocess.PIPE, stderr=subprocess.PIPE,
                    universal_newlines=True, timeout=3600)
print(r.stdout + r.stderr)
```

Example Output (Console Log)

```
18 *Verdi* : Enable RPC Server(15434)
19
20 srctBInvokeSim
21 Verdi>run
22 === Starting Simulation ===
23 Test No. 1 - passed!!
24 Test No. 2 - passed!!
25 Test No. 3 - passed!!
26 Test No. 4 - passed!!
27 Test No. 5 - passed!!
28 Test No.: 10
29 General setup OK address: 00
30 General setup OK address: 01
31 General setup OK address: 02
32 General setup OK address: 03
33 General setup OK address: 04
34 Test 10: Pass - Location 0x00 - OK: Write data = deadbeefbad4f00d, Read data = deadbeefbad4f00d
35 Test No.: 11
36 General setup OK address: 00
37 General setup OK address: 01
38 General setup OK address: 02
39 General setup OK address: 03
40 General setup OK address: 04
41 Test 11: Passed!!
42 Test No.: 12
43 General setup OK address: 00
44 General setup OK address: 01
46 General setup OK address: 02
46 General setup OK address: 03
47 General setup OK address: 04
48 Test 12: Passed!!
49 Test No.: 13
50 General setup OK address: 00
```

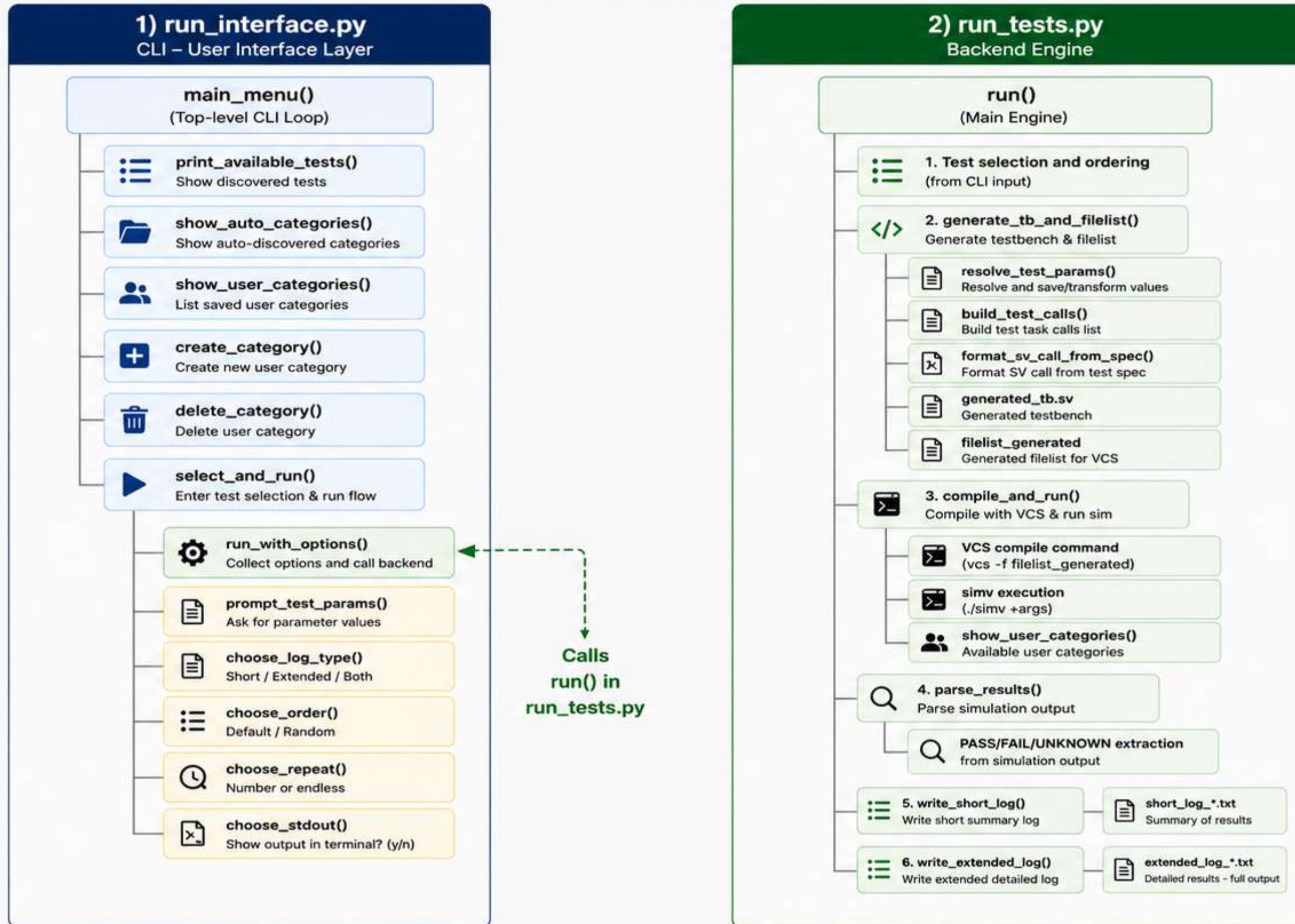

The Automated Verification Architecture



Takeaway: Python manages the state;
SystemVerilog executes the physics.

Python Framework – Final Architecture

Our 2-File Solution: Clean Separation of Interface and Execution





Separate Approach – Feature: Generated SV File & Generated File List

How We Implemented the Solution

- 1 Start from Project Sources**
We use the main testbench (top_iris_tb.sv) and the master file list (filelist_generated).
- 2 Select the Tests to Run**
You choose the specific tests you want to execute.
- 3 Python Generates Tailored Files**
The framework creates a new SV testbench and a matching file list that include only what's needed for your selected tests.
- 4 Run Only What You Need**
Simulation runs faster, is easier to debug, and is fully aligned with your test selection.

1 Project Sources (Inputs)

Open ▾

```
// Top-level Testbench
module top_iris_tb;
...

endmodule
```

Master File List (filelist_generated)

```
/users/shukir/logic_design/Project/.../types_pkg.sv
/users/shukir/logic_design/Project/.../sram_sp_512x64.v
./RTL/access_sm.sv
./RTL/address_decoder.sv
./RTL/all_registers_block.sv
./RTL/clock_divider_1bit.sv
...
./RTL/top_iris.sv
```

2 Your Test Selection

- ☒ Test 10 – Read Dead Beef from Host
- ☒ Test 11 – Read Full Buffer from Host
- ☒ Test 12 – HW Write One Full Buffer
- ☐ Test 13 – HW Write One Partial Buffer
- ☐ Test 14 – HW Write Endless Full Buffer
- ...

3 Python Framework Generates Tailored Files for Your Selected Tests

Generated Testbench (generated_tb.sv)

```
// Auto-generated Testbench
module generated_tb;
...
// Includes only selected tests
// (10, 11, 12)

endmodule
```

Generated File List (auto_filelist.f)

```
/RTL/access_sm.sv
/RTL/address_decoder.sv
/RTL/clock_divider_1bit.sv
/RTL/control_reg_8bits.sv
/RTL/ip_interface.sv
/RTL/memory_on_flops.sv
/RTL/status_reg_8bits.sv
...
/RTL/top_iris.sv
```



Only the files
needed for the
selected tests
are included.

JSON File – Simple and Powerful

One simple file defines exactly what to run.



A simple JSON file captures your test selection.

Our framework reads it and runs exactly the tests you chose – no more, no less.

The Command-Line Interface: Terminal-Based Control

```
epstmh@vlsi-ria68:python_runner$ python3 run_interface.py
```

```
=====
TEST RUNNER - MAIN MENU
=====
```

- 1. Show all available tests
- 2. Show auto-discovered categories
- 3. Show saved user categories
- 4. Create user category
- 5. Delete user category
- 6. Run tests
- 0. Exit

```
-----
Choose: █
```

1. Launch:

Execute `python3 run_interface.py` from the framework directory.

2. Discovery:

Framework parses the SystemVerilog database, auto-discovering and categorizing all available verification tasks.

3. Selection:

Users select tests by broad category, saved user-defined groups, or explicit discrete IDs.

4. Parameter Injection:

System intelligently prompts the user to supply missing manual or random parameter values before execution.

CLI SHOW CASE

GUI Framework



Easy to Use

Simple GUI for selecting and running tests.



Background Logic

Handles the complexity automatically.



Reliable

Consistent and accurate test execution.



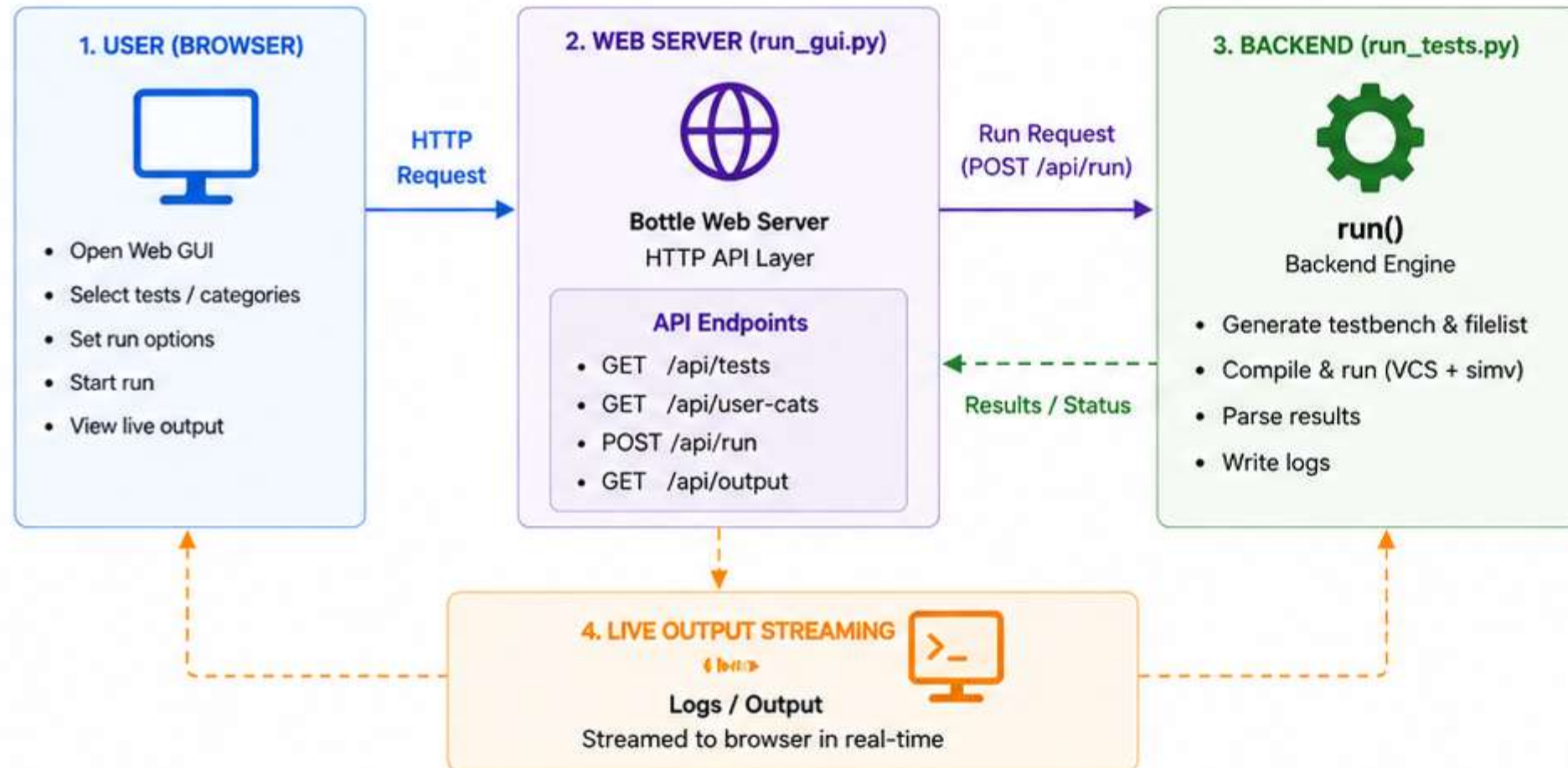
Scalable

Built to grow with new tests and users.

GUI Framework Architecture

A simple and modular framework for test execution, management and live feedback.

run_gui.py – GUI Layer Simple Architecture



>_ Framework Startup (Example)

```
[epstmh@vlsi-ria68 python_runner]$ python3 run_gui.py
```

```
=====
Test Runner GUI
Open in browser: http://localhost:8080
=====
```

```
Bottle v0.12.19 server starting up (using WSGIRefServer())...
Listening on http://127.0.0.1:8080/
Hit Ctrl-C to quit.█
```

 **Open in your browser** <http://localhost:8080>

Legend

-  User Request (Browser → Server)
-  Run Request (Server → Backend)
-  Status / Results (Backend → Server)
-  Live Output Stream (Server → Browser)



User Friendly

Simple web interface for easy test selection and execution.



Modular & Scalable

Separation of GUI, server, backend and streaming for easy maintenance.



Reliable

Consistent execution, result parsing and live feedback.

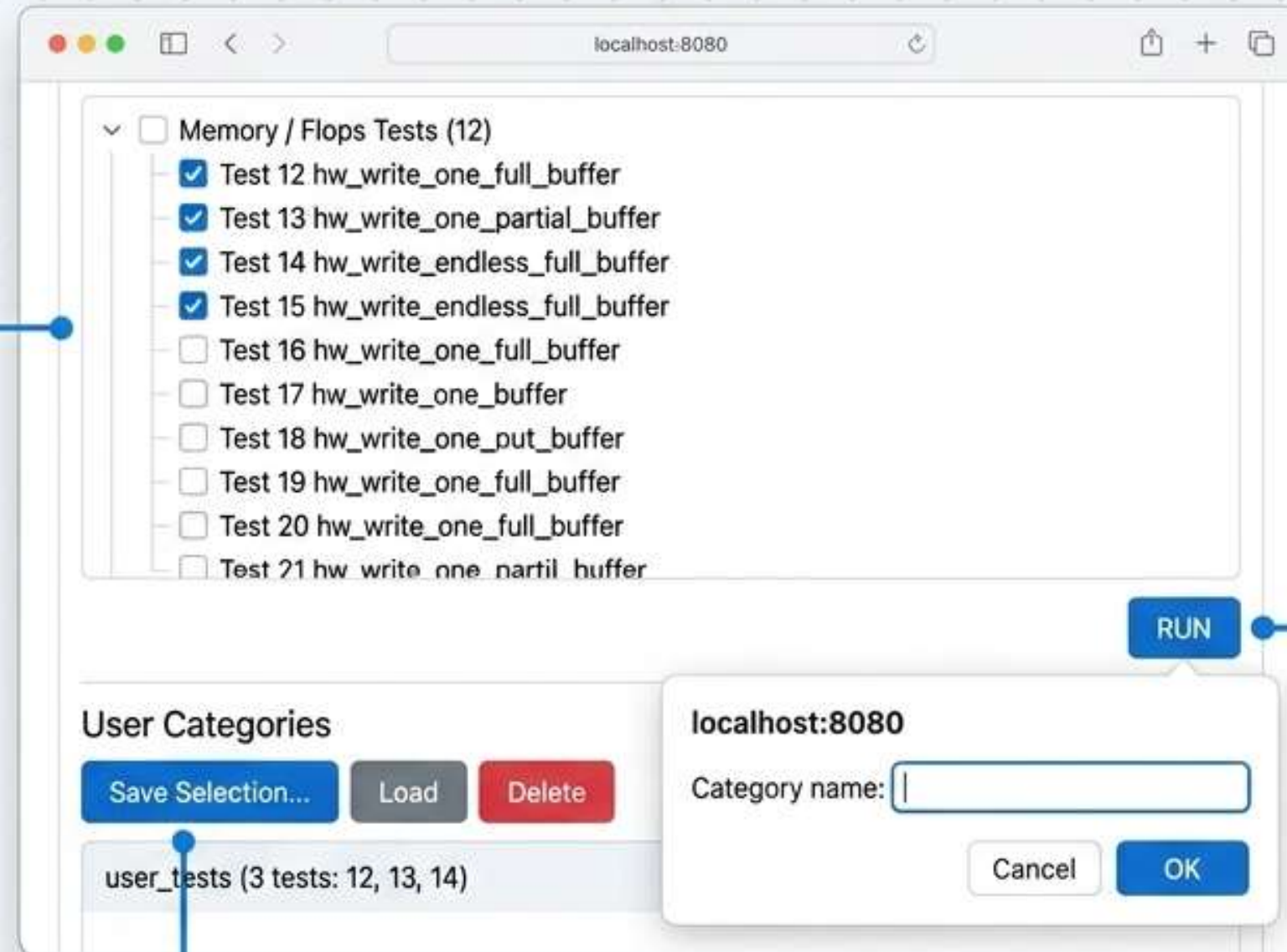


Real-Time Feedback

Live logs and output streamed directly to the browser.

Web GUI: Interactive Test Management

Dynamic Data: GUI retrieves the verification database via `api_tests()`, rendering auto-discovered categories in a visual tree.

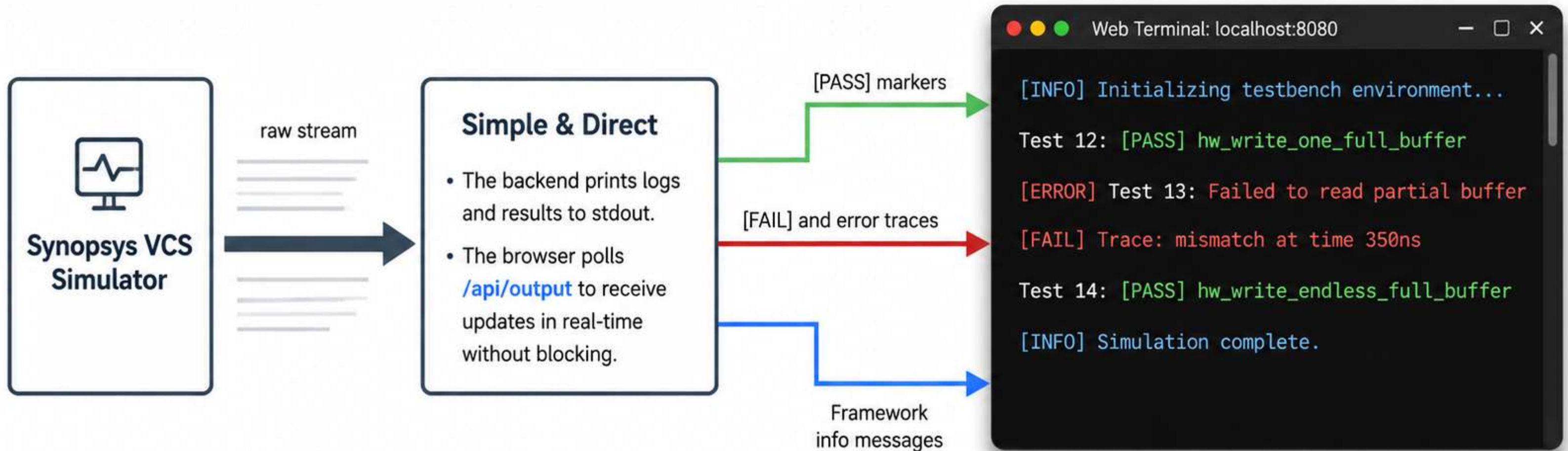


Asynchronous Execution: Hitting 'RUN' sends data to `api_run()`, which launches execution in a background `_run_worker()` thread. The browser never freezes during heavy VCS compilation.

Persistent Regression Sets: Users can group tests and save them to `user_categories.json` for rapid reuse across sessions.

Runtime Visibility: Live Output Streaming

Real-time visibility into simulation progress and results through live output streaming.



Result: Reproduces authentic terminal feedback inside the browser, preserving full backend transparency while running in a background thread.



Real-Time

See results as they happen.



Transparent

No logs are hidden.

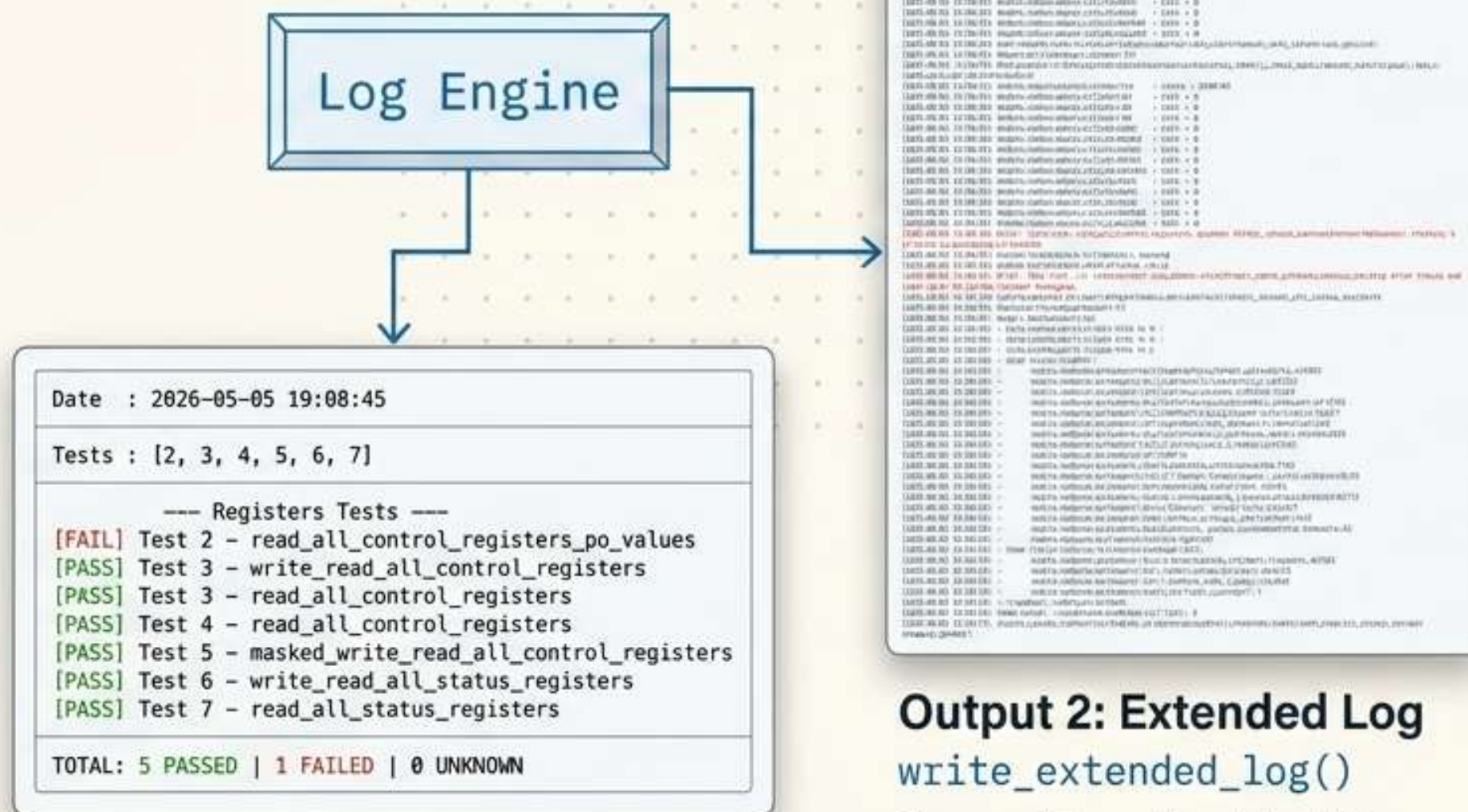


Non-Blocking

UI stays responsive.

Result Parsing and Persistent Logging

Post-Processing:
`parse_results()` scans the raw VCS output to detect pass/fail patterns and map them to explicit Test IDs.



Output 1: Short Log

Output 2: Extended Log

```
write_extended_log()
```

The complete, unaltered simulation output, preserving deep error traces and time steps for granular debugging.

Parameterized Tests

Flexible, reusable and configurable test execution



Parameter Specification

- Each test defines its parameters in a Python configuration (TEST_PARAM_SPECS).
- Includes type, range (min/max), width, and other constraints.
- The GUI reads this to automatically build the input form.



Used in the SystemVerilog Task

- Parameters are declared as inputs to the SystemVerilog task.
- This enables the testbench to pass values to the DUT.



Configurable from the GUI

- The user can set parameter values manually or choose Random.
- GUI enforces valid ranges and parameter types.
- Values are sent to the backend for test execution.



Used During Test Execution

- The selected/Random values are passed to the test call.
- The test runs with these values and results are reported.

```
TEST_PARAM_SPECS = {
    12: {
        "task_name": "hw_write_one_full_buffer",
        "params": [
            {"name": "number_of_cycles", "type": "int", "min": 0, "max": 5000},
        ],
    },
    13: {
        "task_name": "hw_write_one_partial_buffer",
        "params": [
            {"name": "partial_address", "type": "logic", "width": "[10:0]", "min": 10, "max": 2048},
            {"name": "number_of_cycles", "type": "int", "min": "partial_address", "max": 5000},
            {"name": "param", "type": "int", "min": 100, "max": 5000},
        ],
    },
    14: {
        "task_name": "hw_write_endless_full_buffer",
        "params": [
            {"name": "param", "type": "int", "min": 100, "max": 5000},
        ],
    },
}
```

```
task hw_write_ds_one_partial_buffer(
    input logic [10:0] partial_address,
    input int          number_of_cycles,
    input int          param,
    input logic [11:0] ds_number);
    // ...
endtask
```

☐ Test 23 hw_write_ds_one_partial_buffer ▼

partial_address	logic [10:0]	[10..2048]	☐ Random ☒ Manual	465	↕	↕ re-roll
number_of_cycles	int	[465..5000]	☐ Random ☒ Manual	300	↕	
must be in [465..5000] -- try again or pick Random						
param	int	[100..5000]	☐ Random ☒ Manual	4249	↕	↕ re-roll
ds_number	logic [11:0]	[8..4096]	☐ Random ☒ Manual	[8..4096]	↕	

```
// Test No. 13
hw_write_one_partial_buffer(500, 2223, 1757);
```


GUI SHOW CASE

Problems and Solutions

Filelist Generation & Dependency Management



THE CHALLENGE

- Automatically generating the compilation filelist for Synopsys VCS.
- A fully dynamic filelist was attempted to maximize portability between projects.
- Large environments have many project-specific dependencies, compilation orders, include paths, and hierarchy relationships that cannot be inferred automatically.



Incorrect filelist construction can lead to unresolved module errors during compilation.



EXAMPLE – UNRESOLVED MODULE ERRORS

Module definition of above instance is not found in the design.

Error-[URM1] Unresolved modules

I0/pads.sv, 154

```
*PMC_VS_RCD_1033V PAD1_VSS_i{..HVPS_P (HVPS_P), ..HVPS_N (HVPS_N), ..POC_P (POC_P), ..POC_N (POC_N),  
..VREFP (VREFP), ..VREFN (VREFN), ..VDD (VDD)}
```

Module definition of above instance is not found in the design.

Error-[URM1] Unresolved modules

I0/pads.sv, 162

```
*PMC_VS_RCD_1033V PAD5_VSS_i{..HVPS_P (HVPS_P), ..HVPS_N (HVPS_N), ..POC_P (POC_P), ..POC_N (POC_N),  
..VREFP (VREFP), ..VREFN (VREFN), ..VDD (VDD)}
```

Module definition of above instance is not found in the design.

Error-[URM1] Unresolved modules

I0/pads.sv, 166

```
*PMC_VS_RCD_1033V PAD8_VSS_i{..HVPS_P (HVPS_P), ..HVPS_N (HVPS_N), ..POC_P (POC_P), ..POC_N (POC_N),  
..VREFP (VREFP), ..VREFN (VREFN), ..VDD (VDD)}
```

...



FINAL SOLUTION – HYBRID APPROACH

- ✓ Use the original validated project filelist as a reference.
- ✓ Dynamically modify ONLY the relevant testbench entry according to the generated execution environment.
- ✓ Preserves flexibility while maintaining compilation stability and correct dependency resolution across the verification flow.



ROOT CAUSE

- Missing or incorrectly ordered dependencies in the generated filelist.
- VCS cannot locate required module definitions during compilation.
- The framework cannot reliably determine all dependency relationships without prior knowledge of the project structure.

VCS Compilation & Simulation Stability

EXAMPLE – VCS STOPS BEFORE SIMULATION



THE CHALLENGE

- VCS compilation completed successfully but stopped before entering simulation.
- Stopped due to warnings or environment-related issues unrelated to the executed tests.



```
/export/apps/synopsys/vcs/2022.06-SP2/bin/vcs -full64 -sverilog -ntb_ppts uvm-1.2 +v2k -l simv.log -f filelist.f
+define+UVM_NO_DPI +incdir+; -rdynamic -Wl,-rpath,ORIGIN/;/usr/local/lib/synopsys/2022.06-SP2-Full64/bin/23pdc
+snpsmalloc=1:1s -l vcsnew cpro.
/simv up to date
PU time: 1.171 seconds to compile + .574 seconds to elab + .123 seconds to link

Running: ./simv

Chronologic VCS simulator copyright 1991-2022
Contains Synopsys proprietary information.
Compiler version T-2022.06-SP2_Full64; Runtime version T-2022.06-SP2_Full64; Mar 18 02:26 2026
--- Starting simulation ---

**FATAL***: POC_P is NOT connected, it has to be driven by the special pad!
**FATAL***: POC_N is NOT connected, it has to be driven by the special pad!
**FATAL***: VREFP is NOT connected, it has to be driven by the special pad!
**FATAL***: VREFN is NOT connected, it has to be driven by the special pad!
```



ROOT CAUSE



Failures can originate from multiple layers:

- Python scripts
- Generated files
- Testbench integration
- VCS execution flow



Difficult to pinpoint the exact source of the failure at the beginning of development.



Multiple dependencies between environment configuration and VCS behavior.



SOLUTION – LAYERED APPROACH



Rebuilt the framework gradually in a layered manner.



Verified each level independently.



Integrated the next stage only after validation.



Improved error isolation and reduced debugging complexity.

Generality Versus Project Adaptation



THE CHALLENGE



Balancing framework generality with project-specific customization.



Different projects have different structures, helper tasks, compilation settings, and execution flows.



As described in the user manual, several configurations still need to be adjusted when integrating a new project into the framework.

EXAMPLES OF PROJECT-SPECIFIC ADJUSTMENTS



Project paths



Helper task definitions



VCS compilation settings



Test organization structure



THE SOLUTION – A BALANCED APPROACH



GENERIC CORE

Core execution logic remained generic and reusable across projects.



CONFIGURABLE LAYER

Project-specific behavior was handled through configurable setup sections.



This solution improved portability while keeping the framework practical and stable for real verification environments.

Conclusions and Future Work

Conclusion & Project Outcomes



KEY ACHIEVEMENTS



Developed a complete Python-driven verification framework for the IRIS SystemVerilog environment.



Transitioned from manual VCS execution to automated CLI and Web GUI workflows.



Added dynamic test selection, live output streaming, logging, and parameterized test execution.



Created reusable architecture using JSON configuration, generated testbenches, and generated filelists.



ENGINEERING EXPERIENCE & LESSONS LEARNED



Learned how Python interfaces with Synopsys VCS through subprocess execution and automation flows.



Gained practical debugging experience across Python, SystemVerilog, JTAG, and verification integration layers.



Solved real-world challenges involving compilation stability, dependency management, and scalable architecture design.



Used AI-assisted development tools while understanding the importance of human engineering validation and review.



FINAL IMPACT

The final framework transformed the verification flow into a cleaner, faster, and more user-friendly environment while preserving flexibility for future verification projects and extensions.



PATH TOWARD FULL GENERICITY 2 – PART 1

Making the Runner Project-Agnostic



1. CENTRALIZED CONFIGURATION

- Move all project-specific data to an external JSON/YAML configuration file.
- Includes paths, filelists, testbenches, and environment settings.



2. DYNAMIC TASK HANDLING

- Replace hardcoded SETUP_TASKS with a configurable list.
- Automatically ignore project-specific helper tasks.



3. CUSTOMIZABLE PARSING

- Allow users to define regex patterns for test headers and category comments.
- Support different coding styles and conventions.



4. SIMULATOR ABSTRACTION

- Make simulator commands configurable.
- Keep VCS settings as default example, but allow other simulators.

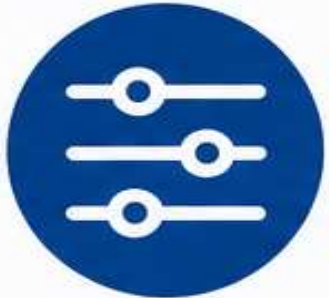
PATH TOWARD FULL GENERICITY 2 – PART 2

Scalability, Portability & Future Enhancements



5. CROSS-PLATFORM SUPPORT

- Ensure compatibility with Windows, macOS and Linux.
- Remove OS-specific assumptions from scripts.



6. ENHANCED PARAMETERIZATION

- Move TEST_PARAM_SPECS to the configuration file.
- Read parameters directly from SystemVerilog task headers.
- Support advanced constraints (e.g. multiple of, unique, ranges).



7. ADVANCED LOGGING & VERBOSITY

- Add VERBOSE flag support inside SV tasks.
- Allow clean logs or detailed debug output.
- Flexible verbosity levels from the configuration.



8. GOAL

- Adapt the runner to any standard SystemVerilog project by editing the configuration file only.
- No changes required in the Python source code.



User Manual

We created a user manual to help anyone use our Python scripts **easily**.

- ✓ Step-by-step guide for setup and usage
- ✓ **No deep knowledge** required
- ✓ Explains the project **structure** clearly
- ✓ Covers **CLI**, **GUI**, **parameters**, **filelist** and more
- ✓ **Examples** and screenshots included
- ✓ Designed for both **beginners** and **advanced** users



Our goal: Make it **simple** for anyone to run and manage **SystemVerilog** tests

USER MANUAL

Python Test Runner



GITHUB & RESOURCES

Repository Link :

https://github.com/samertabri-cpu/VLSI-Project_B-Verification/tree/main

**Thanks
for your attention!**

